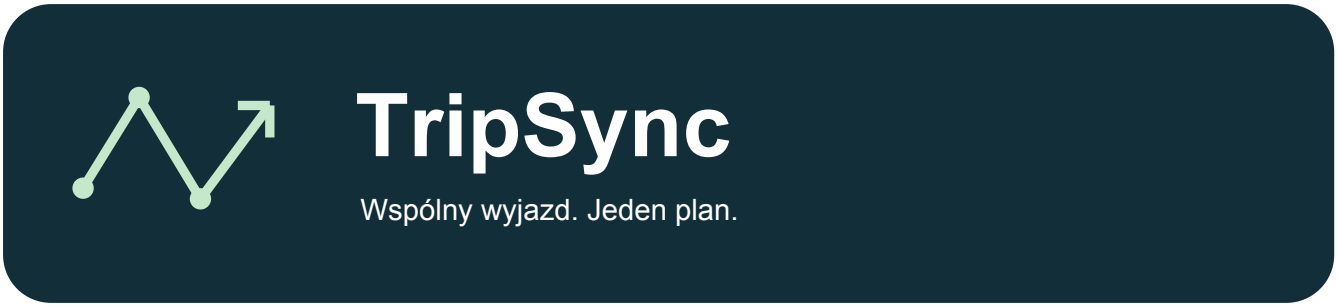


Dokumentacja projektu

Aplikacja Android do wspólnego planowania i rozliczania wyjazdów.

Wersja Android 1.2.0 / kod wersji 3. Backend API 1.1.0. Stan dokumentacji: 05.10.2026.

Projekt dla zespołu trzyosobowego. W obecnym zakresie wszystkie funkcje są bezpłatne; aplikacja nie realizuje płatności ani subskrypcji.

Obszar	Stan projektu
Platforma	Kotlin, Compose / Material 3, MVVM, Android 8.0+
Dane i integracje	Room, REST, MySQL lub SQLite, Open-Meteo i OpenStreetMap
Testy z 05.10.2026	78 różnych testów zaliczonych; lint: 0 błędów, 16 ostrzeżeń
Repozytorium	https://github.com/Yamoteusz/TripSync

Autorzy i zakres oddania

Autor	Imię i nazwisko / numer albumu	Rzeczywiście wykonany zakres
1	Do uzupełnienia	Do opisania na podstawie faktycznej pracy
2	Do uzupełnienia	Do opisania na podstawie faktycznej pracy
3	Do uzupełnienia	Do opisania na podstawie faktycznej pracy

Dane autorów wymagają uzupełnienia przed oddaniem. Dokument opisuje rzeczywisty zakres oraz ograniczenia; nie stanowi deklaracji uzyskania konkretnej oceny.

APK debug: output/apk/TripSync-debug.apk. Instrukcje i dokumenty źródłowe znajdują się w repozytorium. SDK 36.1 / target 36: aktualizacja do stabilnego API 37 pozostaje do wykonania.

Spis dokumentacji

01. Cel, zakres i wymagania
02. Funkcje i korzystanie z aplikacji
03. Architektura i synchronizacja
04. Przypadki użycia
05. Bazy danych i model informacji
06. Biblioteki i technologie
07. API REST - kontrakt backendu 1.1.0
08. Instalacja i uruchomienie
09. Wispbyte + MySQL: aktualna konfiguracja
10. Bezpieczeństwo i obsługa błędów
11. Testowanie i wyniki wersji 1.2
12. Zgodność z wytycznymi zajęć Android
13. Foldery i aktualizacje projektu
14. GitHub Desktop i aktualizacje zespołu
15. Prezentacja i obrona projektu
16. Ograniczenia i dalsze prace
17. Najważniejsze widoki aplikacji: cztery strony rzeczywistych zrzutów ekranu.

Jak korzystać z dokumentacji

PDF zawiera treść dokumentów źródłowych Markdown. Nawigacja po rozdziałach jest dostępna w zakładkach PDF. Spis treści prowadzi do rozdziałów PDF. Pozostałe odnośniki otwierają pliki w repozytorium; dostęp zależy od jego widoczności i uprawnień.

README na GitHubie stanowi skrócony opis i punkt wejścia. Dokumenty w docs/ opisują wymagania, implementację, instalację, wyniki testów oraz scenariusz obrony. AUTHORS.md służy do uzupełnienia autorów i ich faktycznego wkładu.

Wyniki testów pochodzą z zapisanych raportów wersji 1.2.0. Starsze dokumenty i zrzuty wersji 1.1 pozostają materiałami archiwalnymi; nie są dowodem aktualnego wyglądu aplikacji.

Cel, zakres i wymagania

Cel projektu

TripSync porządkuje organizację podróży niewielkich grup znajomych: plan, zadania, pomysły i wspólne koszty. Projekt sprawdza projektowanie, implementację, testowanie oraz dokumentowanie aplikacji Android w zakresie dla trzech osób. Stan: Android 1.2.0, backend API 1.1.0. Zakres nie obejmuje monetyzacji.

Role i odbiorcy

Gość może utworzyć konto, zalogować się lub skorzystać z demo. Uczestnik współpracuje przy planie, zadaniach, pomysłach, miejscach i kosztach. Organizator ma dodatkowe uprawnienia do zarządzania wyjazdem, zaproszeniami, uczestnikami i budżetem. System Android wykonuje zadania WorkManager i lokalne powiadomienia. Uprawnienia opisuje macierz w **przypadki-uzycia.md**.

Wymagania funkcjonalne

ID	Wymaganie	Kryterium odbioru
F01	Rejestracja, logowanie, wylogowanie	Walidacja, sesja lokalna i unieważnienie tokenu przy wylogowaniu online
F02	CRUD wyjazdów	Nazwa, cel, daty, waluta i opis; edycja/usunięcie przez organizatora
F03	Uczestnicy i zaproszenia	Kod 12 znaków lub link; rotacja kodu i kontrola dostępu
F04	CRUD harmonogramu	Data w terminie wyjazdu, godzina, miejsce, kolejność i wykonanie
F05	Propozycje i głosy	CRUD propozycji, jeden własny głos na propozycję, cofnięcie i ponowne oddanie
F06	Propozycja do planu	Po potwierdzeniu formularza powstaje punkt i znika propozycja
F07	Zadania i pakowanie	CRUD, kategoria, osoba, wykonanie i opcjonalne przypomnienie
F08	Wydatki	CRUD, płatnik, data, uczestnicy, podział równy lub kwoty własne
F09	Rozliczenia	Dokładność groszy, suma sald zero i propozycje przelewów
F10	Offline i trwałość	Lokalny zapis bez sieci, ponowny odczyt po uruchomieniu
F11	Synchronizacja	Kolejka, ponowienia bez duplikatów, wersje oraz wybór wersji przy konflikcie
F12	Powiadomienia lokalne	Przypomnienie punktu/zadania, anulowanie po wykonaniu/usunięciu
F13	Stan i ustawienia	Zachowanie formularza po odtworzeniu aktywności, motyw i adres API w DataStore
F14	Obsługa błędów	Walidacja, komunikaty sieci/serwera i stan odrzuconych operacji
F15	Demo i udostępnianie	Edytowalny przykład offline oraz udostępnienie tekstowego planu
F16	Budżet	CRUD limitu całkowitego i kategorii przez organizatora
F17	Kategorie i lista kosztów	Sześć kategorii, zestawienie, filtr, wyszukiwanie i sortowanie
F18	Spłaty	CRUD własnych spłat; salda uwzględniają spłatę częściową i nadpłatę
F19	Historia zmian	Autorytatywne wpisy serwera z autorem i czasem, odczyt bez edycji klienta

ID	Wymaganie	Kryterium odbioru
F20	Miejsca	CRUD współrzędnych, mapa wspólna i otwarcie w zewnętrznych mapach
F21	Wyszukiwanie	Geokodowanie miejscowości i zapis wyniku jako miejsce
F22	Prognoza	7 dni, temperatury, opis oraz szansa opadów
F23	Cache pogody	Ostatni wynik i daty dostępne po błędzie sieci
F24	Daty i godziny	Kalendarz i zegar 24-godzinny; w bazie ISO, prezentacja po polsku
F25	Ochrona formularza	Potwierdzenie odrzucenia zmian w głównym edytorze przy cofnięciu
F26	Widoki wyjazdów i planu	Wyszukiwanie, aktywne/zakończone wyjazdy, plan według dni

Wymagania niefunkcjonalne

ID	Wymaganie	Realizacja i granice
N01	Material 3 i responsywność	Compose, motywy, przewijanie, maksymalna szerokość i kompaktowe pickery; szeroka weryfikacja tabletów pozostaje do wykonania
N02	Architektura i modularność	MVVM, foldery ui/domain/data i odrębny backend; jedna aplikacja Gradle, nie pełna Clean Architecture
N03	Asynchroniczność	Coroutines, Flow, suspend Room/Retrofit i I/O poza głównym wątkiem
N04	Zmiana konfiguracji	ViewModel i rememberSaveable; test odtworzenia aktywności, bez pełnej macierzy wszystkich konfiguracji
N05	Ochrona danych	Keystore, HTTPS, kontrola dostępu i walidacja; baza Room bez SQLCipher
N06	Dostępność	Etykiety pól i ikon funkcyjnych, kontrast i większa czcionka; audyt TalkBack pozostaje do wykonania
N07	Testy	Domena Kotlin, backend, Compose UI i integracja REST
N08	Rozbudowa	Typowane modele, wersjonowane operacje, repozytorium i eksport schematu Room
N09	Git i dokumentacja	Historia, README, diagramy, testy, screeny, PDF i APK

Dowody i status wymagań prowadzącego: **zgodnosc.md**. Ograniczenia i funkcje przyszłe: **ograniczenia.md**. Dane autorów oraz rzeczywisty wkład: **AUTHORS.md**.

Funkcje i korzystanie z aplikacji

Początek i konta

Gość wybiera rejestrację, logowanie albo demo. Demo zawiera wyjazd trzech osób; można go edytować offline. Rejestracja na publicznym lub własnym API tworzy odrębne konto. E-mail jest identyfikatorem, bez wysyłania potwierdzenia. Adres API zmienia się przed logowaniem. Wylogowanie czyści lokalną bazę i niewysłane zmiany; kolejne demo odtwarza przykład.

Wyjazdy i plan

Lista umożliwia wyszukiwanie po nazwie i miejscu oraz filtrowanie aktywnych i zakończonych podróży. Organizator ustala daty, miejsce docelowe, opis i walutę. Podsumowanie pokazuje grupę, koszty, zadania i najbliższy przyszły niewykonany punkt według daty i godziny.

Plan jest pogrupowany według dni. Formularz używa kalendarza ograniczonego do terminu wyjazdu i zegara 24-godzinnego. Na kompaktowym ekranie albo przy większej czcionce pojawia się systemowy kalendarz oraz specjalizowany formularz godziny. Punkt może zawierać zapisane miejsce, opis i przypomnienie. Główny edytor pyta o odrzucenie niezapisanych zmian przy cofnięciu.

Zadania i pomysły

Dwie listy obejmują zadania oraz pakowanie. Element można przypisać osobie i oznaczyć jako wykonany. Pomysły mają tytuł, miejsce i opis. Każdy uczestnik oddaje i cofa własny głos. Przeniesienie propozycji do harmonogramu wymaga potwierdzenia danych punktu w formularzu.

Wydatki, budżet i spłaty

Wydatek ma kwotę, datę, płatnika i mapę udziałów osób. Podział równy rozdziela pozostałe grosze deterministycznie; podział własny wymaga sumy udziałów równej kosztowi. Kwoty są całkowite w groszach/centach. Nowy wydatek otrzymuje dzisiejszą datę.

Sześć kategorii to transport, noclegi, jedzenie, atrakcje, zakupy i pozostałe. Koszty można wyszukiwać po nazwie, kategorii i płatniku, filtrować i sortować. Filtry i przewinięcie zakładek wracają w bieżącej sesji.

Organizator ustala budżet i limity kategorii. Suma limitów nie może przekroczyć całości. Saldo osoby wynosi: zapłacone wydatki minus jej udziały, plus wysłane spłaty, minus odebrane spłaty. Spłatę zapisuje jej nadawca i tylko on może ją zmieniać lub usunąć. Data nie może pochodzić z przyszłości. To deklaracja, nie transakcja bankowa; nadpłata może odwrócić należność.

Miejsca, mapa i pogoda

Miejsce można dodać ręcznie współrzędnymi lub wybrać miejscowość z wyszukiwarki Open-Meteo. Lista jest współdzielona; na mapie pojawiają się znaczniki i atrybucja OpenStreetMap. Mapa pokazuje ładowanie, błąd i ponowienie. Przejście do zewnętrznych map nie wymaga od aplikacji uprawnienia GPS.

Prognoza obejmuje najbliższe 7 dni dla miejsca: minimum i maksimum temperatury, opis i prawdopodobieństwo opadów. Ostatni wynik jest przechowywany lokalnie i oznaczony czasem pobrania. Nie jest to prognoza na dowolnie odległą datę wyjazdu.

Współpraca i błędy

Organizator udostępnia 12-znakowy kod albo link **tripsync://join?code=KOD**. Pozostali logują się na tym samym API. Zapis offline pozostawia operację w kolejce. Ręczna synchronizacja lub WorkManager wysyła zmiany i pobiera dostępne wyjazdy. Konflikt wersji wymaga wyboru danych lokalnych albo serwera. Historia jest dopisywana przez backend po zaakceptowanej zmianie i pokazuje do 200 najnowszych wpisów.

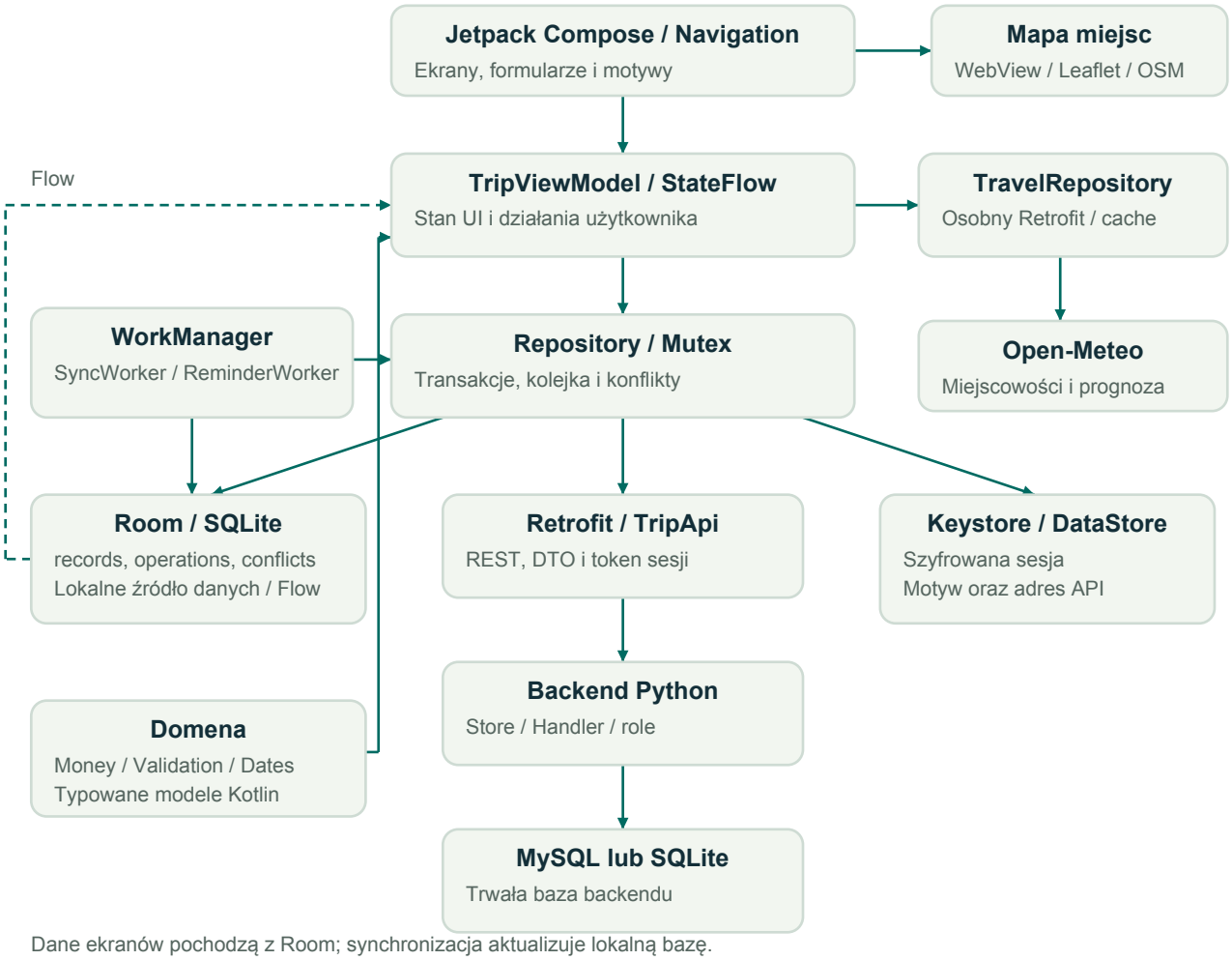
Usunięcie uczestnika z aktywnymi kosztami/spłatami wymaga wcześniejszego uporządkowania tych danych. Kod zaproszenia można unieważnić przez rotację. Nie ma funkcji zmiany właściciela wyjazdu.

Architektura i synchronizacja

MVVM

TripSync wykorzystuje MVVM z wydzielonymi folderami domeny i danych. Jest jedną aplikacją Gradle, z osobnym backendem Python. Nie deklarujemy pełnej Clean Architecture ani osobnych modułów Gradle dla każdej funkcji.

- UI: ekrany Compose, formularze, Theme i Navigation w **ui/**.
- ViewModel: **TripViewModel** zbiera dane repozytorium i publikuje **AppState**, stan błędów, zajętości i pogody przez StateFlow.
- Domena: typowane modele, Money, Validation, Categories i Dates.
- Dane: Room, Repository, TripApi, TravelRepository, Preferences i SessionVault.
- Backend: Handler/Store, autoryzacja, wersjonowanie i adapter SQLite lub MySQL.



Dane ekranów pochodzą z Room; synchronizacja aktualizuje lokalną bazę.

Wersja ilustracyjna: diagram SVG.

Lokalny zapis i przepływ danych

Źródłem danych ekranów jest Room. Formularz waliduje wejście; repozytorium zapisuje rekord oraz operację oczekującą w jednej transakcji. DAO emituje Flow, a ViewModel aktualizuje StateFlow. Ekran zbiera go zgodnie z cyklem życia. Operacje Room/Retrofit są asynchroniczne; pliki i szyfrowana sesja są obsługiwane poza głównym wątkiem.

Mutex repozytorium porządkuje zapisy, logowanie, wylogowanie i synchronizację. Odtworzenie aktywności zachowuje ViewModel i wybrane dane formularza przez rememberSaveable. Dane trwałe pozostają w Room; filtry i przewinięcie zakładek są dodatkowo pamiętane w ViewModel w bieżącej sesji.

Protokół synchronizacji

1. Rekord lokalny i operacja mają identyfikatory, typ, payload i bazową wersję serwera. Ponowna edycja rekordu zastępuje oczekującą operację nowym ID, zachowując bazową wersję.
2. Klient wysyła maksymalnie 100 operacji w batchu, według zależności: wyjazd, propozycja, pozostałe dane, głos.
3. Serwer sprawdza token, dostęp, rolę, dane i **baseVersion**. Poszczególne operacje są odrębnymi transakcjami, nie jedną transakcją całej paczki.
4. Zaakceptowany zapis zwiększa wersję i dopisuje historię w tej samej transakcji. Ponowienie tej samej operacji z tą samą treścią zwraca poprzedni wynik bez duplikatu.
5. Inna wersja bazowa tworzy konflikt. Operacja oczekuje na decyzję o danych lokalnych lub serwera; nie jest automatycznie nadpisywana.
6. Snapshot obejmuje wszystkie dostępne wyjazdy, bez paginacji. Uzupełnia Room i zachowuje niewysłane edycje lokalne.
7. Usunięcie jest przesyłane jako tombstone. Odebranie dostępu usuwa lokalny wyjazd, kolejkę, konflikty i przypomnienia po udanej synchronizacji.

Jeśli odpowiedź serwera zaginie, a klient zmieni rekord ponownie, konflikt może dotyczyć jego własnej wcześniej przyjętej edycji. Mechanizm zachowuje dane, ale wymaga rozstrzygnięcia.

Praca w tle i integracje

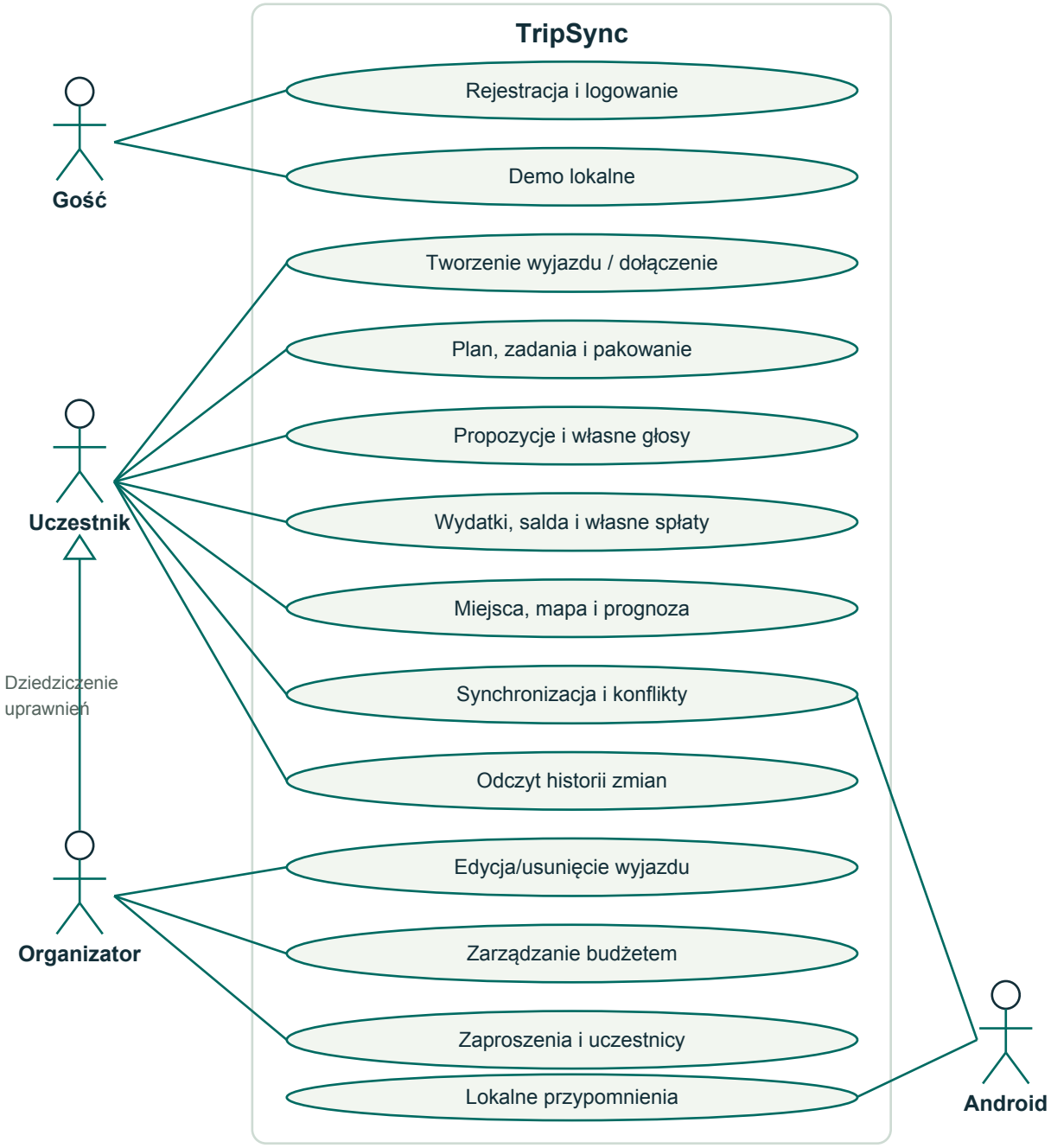
SyncWorker ma warunek sieci; okresowe zadanie ma minimalny interwał 15 minut i zależy od Androida. ReminderWorker obsługuje lokalne przypomnienia, bez gwarancji dokładnej godziny.

TravelRepository używa osobnego Retrofit bez tokenu TripSync. Prognoza jest cache'owana lokalnie, a zmiana miejsca anuluje poprzednie żądanie. Pogoda nie trafia do współdzielonej kolejki. Mapa jest własnym HTML w WebView, z lokalnym Leaflet i kafelkami HTTPS.

WSGI oraz samodzielny serwer używają wspólnej logiki Store/Handler. Hostowana konfiguracja tworzy Store z adapterem MySQL albo SQLite. Operacje backendu są porządkowane przez blokadę jednego procesu; skalowanie na wiele procesów nie zostało zweryfikowane.

Przypadki użycia

Diagram UML



Aktorzy znajdują się poza granicą systemu. Organizator jest specjalizacją uczestnika i dziedziczy jego przypadki użycia; dodatkowo zarządza wyjazdem, grupą i budżetem. System Android realizuje zadania w tle. Diagram grupuje powiązane działania CRUD dla czytelności; macierz poniżej precyzuje dostęp.

Macierz uprawnień

Działanie	Gość	Uczestnik	Organizator
Konto / demo	Tak	Po wylogowaniu	Po wylogowaniu

Działanie	Gość	Uczestnik	Organizator
Utworzenie wyjazdu	Nie	Tak, staje się organizatorem nowego wyjazdu	Tak
Edycja/usunięcie istniejącego wyjazdu	Nie	Nie	Tak
Dołączenie kodem/linkiem	Nie	Tak	Tak
CRUD planu, zadań, propozycji, miejsc i kosztów	Tylko lokalne demo	Tak	Tak
Głosowanie	Tylko demo	Za siebie	Za siebie
Spłata: zapis/edycja/usunięcie	Tylko demo	Własna	Własna
Zapis/edycja/usunięcie budżetu	Tylko demo	Nie	Tak
Odczyt budżetu, historii i danych wyjazdu	Tylko demo	Tak	Tak
Kod zaproszenia i jego rotacja	Nie	Nie	Tak
Usunięcie uczestnika	Nie	Nie	Tak, z ograniczeniami finansowymi
Synchronizacja / konflikty	Nie w demo	Tak	Tak

Rola jest przypisana do konkretnego wyjazdu, nie globalnie do konta. Uprawnienia są egzekwowane przez backend; demo działa lokalnie jako organizator przykładu.

UC01 - Utworzenie i udostępnienie wyjazdu

Warunek: użytkownik jest zalogowany. Tworzy wyjazd, wybiera daty oraz walutę, zapisuje i synchronizuje. Backend tworzy członkostwo organizatora. Użytkownik otwiera uczestników i pobiera kod. Druga osoba dołącza, synchronizuje i widzi dane. Nieprawidłowy kod, brak sesji lub sieci daje komunikat; lokalny zapis wyjazdu może oczekiwać w kolejce przed wygenerowaniem zaproszenia.

UC02 - Zapis planu offline

Warunek: wyjazd znajduje się w lokalnej bazie. Użytkownik dodaje punkt z datą w terminie wyjazdu i poprawną godziną. Room zapisuje rekord i operację. Ekran odświeża się bez internetu. Po odzyskaniu sieci synchronizacja przesyła zmianę. Błędne dane są odrzucane; konflikt wersji prowadzi do UC04.

UC03 - Zapis kosztu i spłaty

Uczestnik wpisuje dodatnią kwotę, płatnika i udziały sumujące się do kosztu. Salda aktualizują się lokalnie. Nadawca zapisuje własną spłatę z datą nie późniejszą niż dziś; saldo uwzględnia ten zapis. Nie jest wykonywany przelew bankowy. Edycja cudzej spłaty jest odrzucana na serwerze.

UC04 - Konflikt edycji

Dwie osoby zmieniają ten sam rekord na tej samej wersji bazowej. Serwer przyjmuje pierwszą operację; drugiej zwraca aktualny rekord jako konflikt. Użytkownik wybiera wersję serwera albo ponawia własną na nowej wersji bazowej. Ponowienie może ponownie wejść w konflikt, jeśli nastąpiła kolejna równoczesna zmiana.

UC05 - Mapa i pogoda

Uczestnik zapisuje miejsce lub wybiera miejscowość z wyszukiwarki. Mapa pokazuje znacznik, a prognoza dane dla najbliższych siedmiu dni. Przy braku sieci mapa pokazuje błąd, lista miejsc pozostaje dostępna, a pogoda może wyświetlić oznaczony datą cache. Nie jest pobierany GPS telefonu.

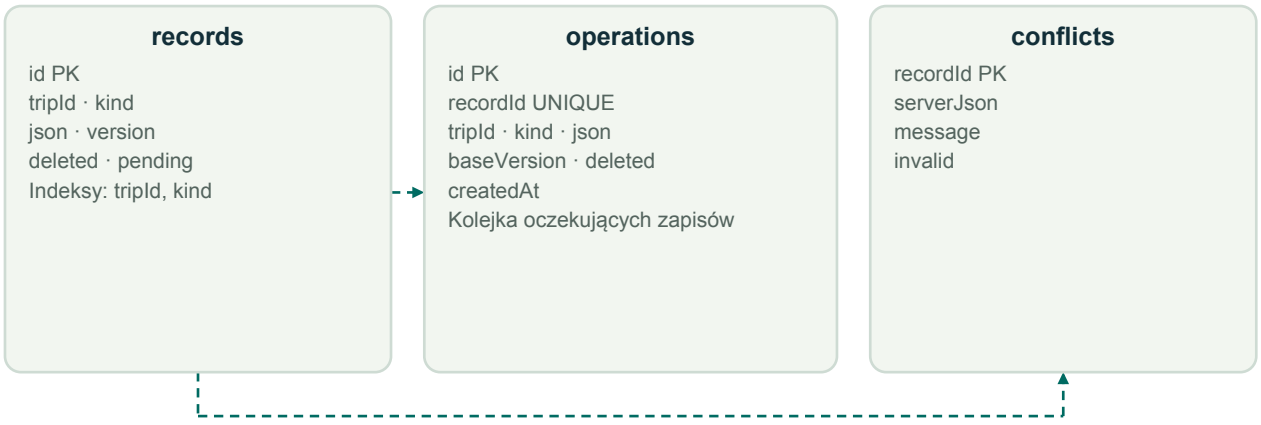
Bazy danych i model informacji

Dlaczego rekordy z payloadem JSON

Wiele typów danych współdzieli identyfikator, wyjazd, wersję i znacznik usunięcia. Jedna tabela rekordów umożliwia jednolity protokół offline i konfliktów. Payloady są reprezentowane typowanymi modelami Kotlin, a spójność referencji jest sprawdzana przez repozytorium i serwer. To świadoma alternatywa wobec osobnej tabeli każdego typu; część relacji nie jest egzekwowana przez FK.

Room na urządzeniu

Schemat wersji 1 znajduje się w **app/schemas/pl.tripsync.data.TripDatabase/1.json**. Wersja Android 1.2 nie zmienia fizycznego schematu.

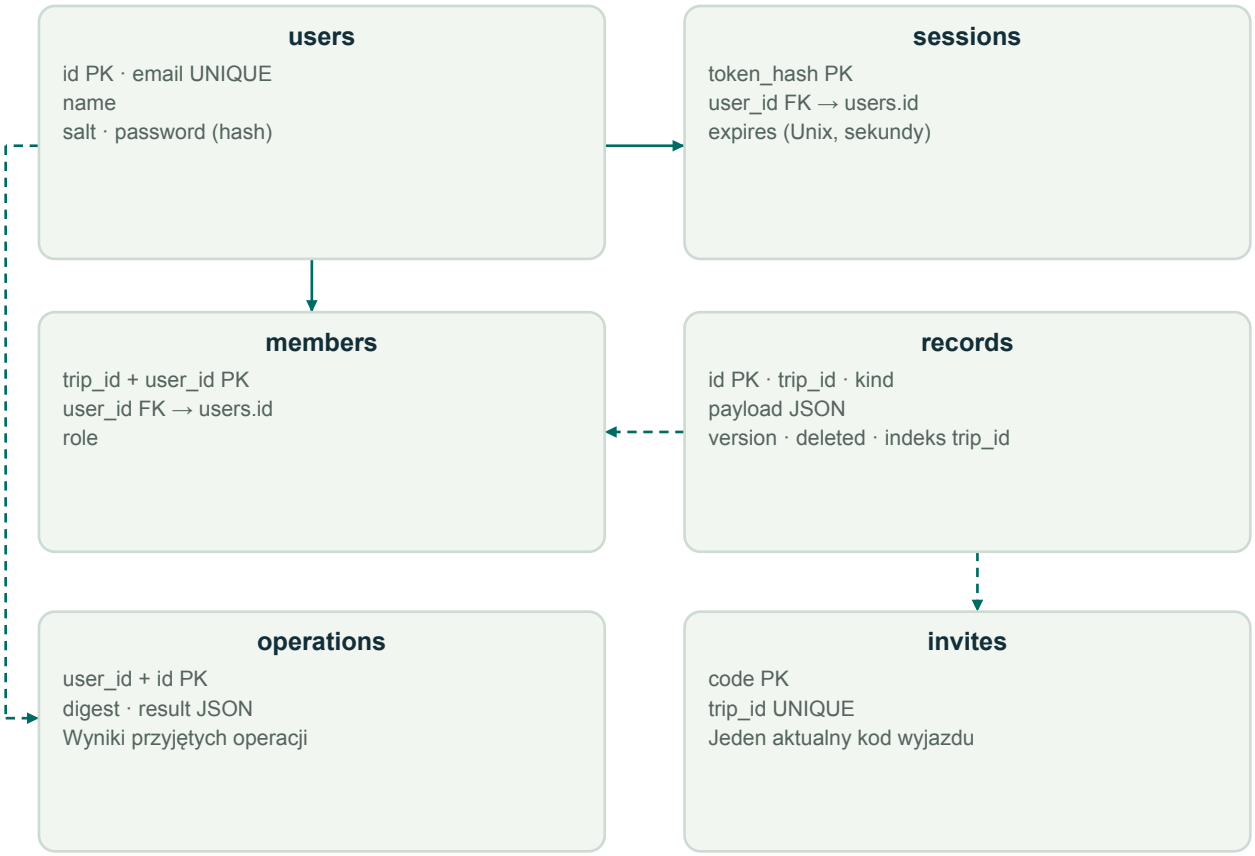


Powiązania logiczne, bez fizycznych FK. Zapis rekordu i operacji w jednej transakcji Room.

Diagram SVG. **records** ma indeksy **tripId** i **kind**. Unikalny indeks **operations.recordId** zapewnia jedną oczekującą operację na rekord. Powiązania diagramu są logiczne; Room nie deklaruje FK pomiędzy tymi trzema tabelami. Zapis rekordu i operacji odbywa się transakcyjnie. Konflikt przechowuje dane serwera i komunikat.

Backend SQLite / MySQL

Oba warianty mają sześć tabel. MySQL używa InnoDB, utf8mb4 i collation utf8mb4_bin; adapter tłumaczy parametry zapytań i operację zastępującą rekord. SQLite włącza FK i WAL. Adapter nie migruje automatycznie istniejącej SQLite do MySQL.



Linia ciągła: fizyczna relacja FK. Linia przerywana: relacja logiczna kontrolowana w kodzie.
MySQL: InnoDB / utf8mb4. SQLite: FK włączone / WAL. Typy szczegółowe: baza-danych.md.

Diagram SVG. Typ **text** na diagramie jest opisem wspólnego modelu; MySQL stosuje ograniczone VARCHAR, LONGTEXT, BIGINT i TINYINT według **backend/mysql_database.py**. Pole **password** zawiera hash, nie hasło jawne. **expires** to czas Unix w sekundach; czas historii **at** jest w milisekundach.

Fizyczne FK: **sessions.user_id** i **members.user_id** do **users.id**. Relacje do wyjazdu w **records**, operacji użytkownika i pól payloadu są logiczne. Tabela **operations** backendu przechowuje wyniki już obsłużonych operacji dla idempotencji; lokalna tabela o tej samej nazwie jest kolejką oczekującą. Nie należy utożsamiać ich funkcji.

Payloady rekordów

kind	Model / dane
trip	TripData: tytuł, cel, daty, waluta, opis
member	MemberData: userId, nazwa i rola
event	EventData: tytuł, data, godzina, miejsce, opis, rank, wykonanie i reminder
task	TaskData: kategoria, assignee, termin, wykonanie i reminder
proposal	ProposalData: tytuł, miejsce i opis
vote	VoteData: proposalId i userId
expense	ExpenseData: kwota, payer, shares, data i category
budget	BudgetData: limit i categoryLimits

kind	Model / dane
settlement	SettlementData: from, to, amount, data i note
place	PlaceData: tytuł, latitude, longitude i opis
activity	ActivityData: autor, akcja, rodzaj, tytuł i czas serwera

Wyjazd ma **id=tripId**, członkostwo **id=tripId:userId**, głos **id=proposalId:userId**, budżet **id=budget:tripId**. Pozostałe rekordy otrzymują ID. Daty kalendarzowe są ISO **YYYY-MM-DD**; godziny to **HH:mm**. Kwoty to całkowite grosze/centy. Kategoria nieobecna w starszym koszcie jest interpretowana jako **other**.

Integralność i trwałość

Serwer sprawdza istnienie powiązanych osób i wyjazdu, własność głosu/spłaty, sumę udziałów oraz ograniczenia waluty. Waluty nie można zmienić po zapisaniu aktywnych kosztów, budżetu lub spłat. Usunięcie użytkownika z wyjazdu z aktywnymi powiązaniami finansowymi jest ograniczone. Zniknięcie uprawnień jest odtwarzane na urządzeniu po synchronizacji.

Zapis JSON rozszerza modele bez zmiany schematu Room; każda przyszła zmiana fizycznego schematu wymaga osobnej strategii migracji. Lokalna baza nie jest szyfrowana SQLCipher. Kopie SQLite wykonuje **backend/backup.py**; MySQL wymaga eksportu bazy na hostingu.

Biblioteki i technologie

Wersje poniżej są deklarowane w konfiguracji Gradle bieżącego projektu. Gradle może podnieść wersje przechodnich zależności podczas rozwiązywania grafu; deklaracja BOM nie oznacza, że każdy artefakt Compose ma dokładnie wersję z tego BOM. Konfiguracja źródłowa jest punktem odniesienia.

Technologia	Deklaracja / wersja	Zastosowanie
Kotlin	Wbudowany Kotlin AGP; plugin Compose 2.2.10	Android, modele i domena
Android Gradle Plugin	9.1.1	Budowanie aplikacji
Gradle Wrapper	9.3.1	Powtarzalne uruchamianie Gradle
Android SDK	compile 36.1, target 36, min 26	Kompilacja i zgodność Android 8.0+
Java / JDK	Kod Java 17; sprawdzono JDK 21	Narzędzia budowania
Compose BOM	2024.01.00	Deklarowana platforma zależności Compose
Compose / Material 3	Wersje zarządzane grafem Gradle	Ekrany, motywy i kontrolki
Activity Compose	1.13.0	Aktywność, stan i obsługa Wstecz
Lifecycle	2.9.0	ViewModel i zbieranie stanu zgodnie z cyklem życia
Navigation Compose	2.7.7	Nawigacja wieloekranowa
Room	2.8.4	SQLite lokalne, DAO, transakcje i Flow
KSP	2.2.10-2.0.2	Generowanie implementacji Room
DataStore	1.1.7	Motyw i adres API
WorkManager	2.9.0	Okresowa synchronizacja i przypomnienia
Coroutines	1.10.2	Asynchroniczność, Flow i współbieżność
Retrofit / converter-gson	2.9.0	Klienci REST i DTO
OkHttp	4.12.0	Transport HTTP
Gson	2.11.0	Kodowanie payloadów i DTO
Android Keystore / AES-GCM	API Android	Szyfrowana sesja lokalna
Python	3.10+; lokalnie testowano 3.13	Backend REST i testy
SQLite	Biblioteka standardowa Python	Lokalny backend i testy
PyMySQL	1.2.3, wariant rsa	Adapter bazy MySQL na hostingu
Leaflet	1.9.4	Biblioteka map dołączona lokalnie w assets
OpenStreetMap	Kafelki HTTPS	Obraz mapy i atrybucja
Open-Meteo	Geocoding / forecast API	Miejscowości i 7-dniowa pogoda
JUnit	4.13.2	Testy jednostkowe Kotlin
AndroidX Test	runner 1.7.0, ext-junit 1.3.0	Testy urządzeniowe
Espresso	3.7.0	Test kompaktowego systemowego kalendarza

Technologia	Deklaracja / wersja	Zastosowanie
Compose UI Test	Graf Compose	Testy formularzy i nawigacji
ReportLab	Narzędzie generatora dokumentacji	PDF, bez zależności aplikacji od Pythona

Licencje i źródła

Leaflet ma licencję BSD-2-Clause dołączoną w **app/src/main/assets/leaflet/LICENSE**. Mapa wyświetla atrybucję OpenStreetMap. Warunki usług Open-Meteo i kafelków trzeba respektować także w projekcie bezpłatnym; nie pobieramy całych obszarów map offline. Przed komercyjną rozbudową trzeba ponownie sprawdzić warunki dostawców.

- Compose i BOM
- MVVM / architektura Android
- Room
- WorkManager
- Android Keystore
- Retrofit
- Leaflet
- Open-Meteo
- Polityka kafelków OSM

Aktualizacja bibliotek i SDK jest odrębną zmianą kodu. Dokumentacja nie deklaruje, że wszystkie zależności są najnowsze.

API REST - kontrakt backendu 1.1.0

Android 1.2.0 zachowuje kontrakt API 1.1.0. Endpoint **/health** raportuje wersję backendu, a nie interfejsu.

JSON UTF-8; wszystkie endpointy poza health i uwierzytelnianiem wymagają **Authorization: Bearer TOKEN**. Odpowiedź błędu HTTP zawiera **message**. Limit treści żądania: 256 KiB. Żądania nie mają interfejsu CORS dla przeglądarki; klientem jest aplikacja Android.

Metoda	Ścieżka	Dane / wynik
GET	**/health**	**status**, **version**
POST	**/auth/register**	**name**, **email**, **password** → **token**, **user**
POST	**/auth/login**	**email**, **password** → **token**, **user**
POST	**/auth/logout**	Unieważnia bieżącą sesję
GET	**/sync**	**tripIds**, **records** dostępnych wyjazdów
POST	**/operations**	**operations** → **results**; od 1 do 100 operacji
GET	**/trips/{id}/invite**	Kod zaproszenia organizatora
POST	**/trips/{id}/invite**	Nowy kod; poprzedni traci ważność
POST	**/join**	**code** → **tripId**

Operacja

```
{
  "id": "uuid-operacji",
  "recordId": "uuid-rekordu",
  "tripId": "uuid-wyjazdu",
  "kind": "event",
  "payload": {
    "title": "Spacer", "date": "2026-10-11", "time": "09:00",
    "place": "Zakopane", "description": "", "rank": 0,
    "done": false, "reminder": false
  },
  "baseVersion": 0,
  "deleted": false
}
```

Wyniki: **applied** z aktualnym rekordem, **conflict** z wersją serwera, **forbidden** lub **invalid** z komunikatem. Wynik operacji jest niezależny od pozostałych elementów batcha; nie jest to transakcja całej paczki. Rekord ma pola **id**, **tripId**, **kind**, **payload**, **version**, **deleted**.

Rodzaje rekordów: trip, member, event, task, proposal, vote, expense, budget, settlement, place oraz generowane przez serwer activity. Wyjazd ma **id=tripId**. Uczestnik ma **id=tripId:userId**. Głos ma **id=proposalId:userId**. Wydatki używają całkowitych kwot w groszach/centach i mapy **shares** od identyfikatora użytkownika do jego udziału.

member powstaje wyłącznie po utworzeniu wyjazdu lub dołączeniu przez zaproszenie. API operacji dopuszcza tylko usunięcie uczestnika przez organizatora. Nie ma endpointu zmiany właściciela.

Payloady obsługiwane przez Android 1.2

- **expense**: dodatkowe opcjonalne **category**: transport, accommodation, food, activities, shopping, other. Starszy rekord bez pola traktowany jest jako other.

- **budget**: stały **recordId=budget:tripId**, **limit** i mapa **categoryLimits** (kwoty całkowite w groszach/centach). Jeden budżet wyjazdu; zapis/usunięcie/przywrócenie tylko przez organizatora. Suma limitów kategorii ≤ limit całkowity.
- **settlement**: **from**, **to**, **amount**, **date**, **note**. Nadawca musi być zalogowanym użytkownikiem i nie może zmienić właściciela istniejącej spłaty. Tylko nadawca edytuje i usuwa. Data nie może być z przyszłości. To deklaracja, bez potwierdzenia bankowego.
- **place**: **title**, **latitude**, **longitude**, **description**. Zakresy współrzędnych [-90,90] i [-180,180].
- **activity**: **actorId**, **actorName**, **action**, **recordKind**, **title**, **at** (czas serwera w ms). Powstaje wyłącznie na serwerze, w transakcji zapisu. Ponowienia i konflikty nie tworzą powtórnych wpisów. Nie przyjmuje operacji klienta i nie zawiera sekretów zaproszeń.

Waluta jest blokowana po dodaniu aktywnych wydatków, budżetu lub spłat. Usunięcie uczestnika z aktywnymi spłatami wymaga ich wcześniejszego usunięcia przez nadawcę.

Zewnętrzne publiczne API

Android używa oddzielnych klientów Retrofit bez tokenu TripSync:

- **https://geocoding-api.open-meteo.com/v1/search?name=...&count=8&language=pl** — miejscowości.
- **https://api.open-meteo.com/v1/forecast?latitude=...&longitude=...&daily=weather_code,temperature_2m_max,temperature_2m_min,precipitation_probability_max&timezone=auto&forecast_days=7** — prognoza.
- **https://tile.openstreetmap.org/{z}/{x}/{y}.png** — kafelki, standardowy cache WebView, identyfikujący User-Agent TripSync/<wersja aplikacji>, widoczna atrybucja, brak masowego pobierania.

Źródła: Open-Meteo, geokodowanie, polityka kafelków. Darmowe Open-Meteo jest przeznaczone do niekomercyjnego użycia i ma limity oraz brak SLA. Współrzędne wybranego miejsca są przekazywane tym dostawcom po użyciu mapy/prognozy; lokalizacja telefonu nie jest pobierana.

Odpowiedzi HTTP

Status	Znaczenie
200	Sukces, także batch z wynikami poszczególnych operacji
400	Błędny JSON lub dane żądania
401	Brak tokenu, błędna lub wygasła sesja
403	Brak wymaganych uprawnień
404	Nieznany endpoint lub zasób
429	Limit prób rejestracji/logowania
503	Chwilowa niedostępność bazy lub I/O
500	Inny błąd serwera, bez ujawniania szczegółów

Status wyniku operacji **conflict** nie jest osobnym HTTP 409: batch odpowiada 200, a klient odczytuje **results[].status**. Autoryzacja całego żądania jest sprawdzana przed obsługą paczki.

Przykład publicznego endpointu bez sekretów:

```
curl https://tripsync.wisp.uno/health
```

Dokładny model rekordu i schemat bazy opisuje **baza-danych.md**.

Instalacja i uruchomienie

Android Studio na Macu i pozostałych systemach

1. Sklonuj repozytorium przez GitHub Desktop lub Git. Otwórz cały katalog z **settings.gradle.kts**, nie sam folder **app**.
2. W Android Studio zainstaluj platformę Android SDK 36.1 i Build Tools 36.0.0. Jest to konfiguracja obecnego kodu, nie deklaracja najnowszego stabilnego SDK.
3. Ustaw Gradle JDK 17 lub 21; lokalnie zweryfikowano JDK 21 z Android Studio. Używaj dołączonego Gradle Wrapper 9.3.1.
4. Android Studio utworzy lokalny **local.properties** z **sdk.dir**. Nie wysyłaj go do Git.
5. Wybierz **File → Sync Project with Gradle Files**. Przy pierwszym pobraniu zależności potrzebny jest internet.
6. Wybierz emulator lub telefon z Androidem 8.0+ i konfigurację **app**, następnie **Run**.

Emulator używany w testach: Android 10 / API 29, ARM64. Minimalny API aplikacji to 26; nie należy go zmieniać tylko po to, aby uruchomić aplikację na nowszym emulatorze.

Gradle i pliki wynikowe

```
./gradlew :app:assembleDebug
```

Na macOS pliki powstają w **~/.gradle/tripsync-builds/<identyfikator-katalogu>/app/**. Przeniesienie projektu do innego katalogu nada osobny identyfikator. Na innych systemach jest to **app/build/**.

Dla dokładnie znanej lokalizacji:

```
./gradlew -PtripsyncBuildRoot=/tmp/tripsync-build :app:assembleDebug
```

APK: **/tmp/tripsync-build/app/outputs/apk/debug/app-debug.apk**. W przypadku konfliktów synchronizacji Dokumentów nie edytuj plików **.class**; są generowane. W tym projekcie lokalizacja poza Dokumentami zapobiega ich użyciu w kolejnych kompilacjach. Nie zmieniaj źródeł na podstawie dekompilowanych klas.

Pełny skrypt weryfikacji i kopiowania APK:

```
sh scripts/build.sh
```

Skrypt używa katalogu tymczasowego lub zmiennej **TRIPSYNC_BUILD_ROOT** i zapisuje końcowy APK w **output/apk/TripSync-debug.apk**.

Instalacja APK na telefonie

Zainstaluj APK z lokalnej paczki lub artefaktu udanego GitHub Actions. Przy instalacji ręcznej Android może wymagać jednorazowej zgody dla źródła pliku. Przy urządzeniu USB:

```
adb install -r output/apk/TripSync-debug.apk
```

W przypadku kilku urządzeń wybierz jedno: **adb -s IDENTYFIKATOR install -r ...**. Aktualizacja zachowuje dane, jeśli pakiet i podpis są zgodne. APK debug z innego komputera może mieć inny klucz: nie odinstalowuj aplikacji z ważnymi niewysłanymi danymi wyłącznie po to, aby obejść błąd podpisu.

Demo i prawdziwe konta

Wypróbuj bez konta tworzy lokalny przykład. Demo nie wysyła danych do serwera. Wylogowanie czyści lokalne dane i niewysłane zmiany; ponowne wejście do demo odtwarza przykład.

Nowa instalacja używa **https://tripsync.wisp.uno/**. Dotychczas zapisany adres jest zachowany. Adres można zmienić w ustawieniach po wylogowaniu, przed utworzeniem konta. Każda osoba korzysta z własnego konta na tym

samym serwerze; nie używa hasła bazy MySQL.

Własny backend lokalny

Python 3.10+. Wariant SQLite korzysta z biblioteki standardowej:

```
python3 backend/server.py --db /tmp/tripsync-local.sqlite
```

Domyślny port to 8080. W osobnym terminalu:

```
curl http://127.0.0.1:8080/health
```

Odpowiedź bieżącego backendu: **{"status":"ok","version":"1.1.0"}**. Numer wersji API nie jest numerem wersji interfejsu Android 1.2.0; kontrakt nie zmienił się przy przebudowie UI.

Ustaw w emulatorze debug **http://10.0.2.2:8080/**. Telefon przez USB może użyć:

```
adb reverse tcp:8080 tcp:8080
```

Wtedy adres aplikacji to **http://127.0.0.1:8080/**. Lokalny HTTP jest dopuszczony tylko w debug dla **10.0.2.2**, **127.0.0.1** i **localhost**. Udostępnienie innym urządzeniom wymaga HTTPS z zaufanym certyfikatem; instrukcja hostingu: wispbyte.md.

Diagnoza najczęstszych problemów

Objaw	Sprawdzenie
Nie ma konta po wejściu do demo	Demo jest odrębne i lokalne; użyj rejestracji na wybranym API
Brak synchronizacji	Adres API, internet, stan kolejki oraz komunikat sesji/konfliktu
defined multiple times i nazwy **2.class**	Zsynchronizuj aktualny Gradle; generowane pliki na Macu powinny być poza Dokumentami
Nie działa MySQL	Sprawdź użytkownika bazy, host, hasło i dostęp; używaj panelu Wispbyte, nie ustawień Androida
.env zgłasza numer wiersza	Każdy wpis ma formę **NAZWA=wartość**; samej nazwy bez **=** parser nie zaakceptuje
Mapa lub pogoda nie ładuje się	Dostęp do zewnętrznych HTTPS; zapisane miejsca i cache pogody mogą pozostać dostępne
Brak powiadomienia	Zgoda od Androida 13, włączone przypomnienie oraz ograniczenia pracy w tle

Wispbyte + MySQL: aktualna konfiguracja

To główna instrukcja hostowanego backendu TripSync. Obecny serwer projektu używa domeny **https://tripsync.wisp.uno/**, portu API 9853, Pythona 3.11 i MySQL. Zrzut panelu wskazywał 1 GiB RAM i 10 GiB dysku. Użytkownik potwierdził działanie aplikacji po poprawieniu konfiguracji bazy; nie przedstawiamy tego jako automatycznego testu obciążenia czy audytu hostingu.

Startup: zachowaj domyślną komendę

Nie trzeba zmieniać komendy startowej Wispbyte. Jej ostatni krok uruchamia **/usr/local/bin/python /home/container/main.py**. Root repozytorium zawiera **main.py**, **requirements.txt** i **backend/**. Katalog **/home/container** musi być checkoutem repozytorium, bez dodatkowego nadrzędnego folderu.

W ustawieniach startupu panelu:

Pole / zmienna panelu	Wartość
Python	3.11 lub zgodne 3.10+
REQUIREMENTS_FILE	requirements.txt
PY_FILE, jeśli udostępnione	main.py
AUTO_UPDATE, jeśli udostępnione	1 dla git pull przy starcie
Port aplikacji	Przydzielony przez panel, obecnie 9853

requirements.txt wskazuje zależności MySQL z **backend/requirements-mysql.txt**. Panel instaluje je przed uruchomieniem. Ustawienie REQUIREMENTS_FILE musi być dostępne dla komendy panelu; wpis tylko do pliku czytanego później przez aplikację nie gwarantuje instalacji pakietów przed startem.

Environment albo plik .env

W **Configuration → Environment** można podać zmienne. W pokazanym panelu edytor wskazuje **.env**. Każdy wpis pliku ma format **NAZWA=wartość**; sama nazwa bez **=** jest błędem. Wzór znajduje się w **backend/wispbyte.env.example**, bez hasła.

Zmienna	Znaczenie
TRIPSYNC_DATABASE_ENGINE	mysql
TRIPSYNC_MYSQL_HOST	Host bazy z panelu
TRIPSYNC_MYSQL_PORT	3306 lub port bazy z panelu
TRIPSYNC_MYSQL_DATABASE	Pełna nazwa bazy z panelu
TRIPSYNC_MYSQL_USER	Pełna nazwa użytkownika z panelu, dokładnie z właściwym prefiksem
TRIPSYNC_MYSQL_PASSWORD	Hasło bazy, wyłącznie na serwerze
PORT	9853 lub przydzielony port API
TRIPSYNC_MYSQL_SSL_CA	Opcjonalna ścieżka CA, gdy hosting zapewnia TLS MySQL
TRIPSYNC_TRUSTED_PROXIES	Opcjonalne, potwierdzone adresy IP proxy, rozdzielone przecinkami

Plik **/home/container/.env** znajduje się obok **main.py**. Parser nie usuwa cudzysłowów, spacji ani znaków **$**, **#**, **=** z wartości: są dosłowne. Nie dopisuj komentarza po haśle. Każda wartość zajmuje jeden wiersz. Wpisy nieobsługiwane przez aplikację, np. poprawny **REQUIREMENTS_FILE=requirements.txt**, są pomijane przez

parser aplikacji; nadal mogą służyć panelowi.

Rzeczywiste zmienne środowiskowe mają pierwszeństwo nad plikiem, także jeśli błędną wartość wpisano wcześniej. Pełna konfiguracja MySQL w środowisku pozwala pominąć plik. Hasła nie wpisuj do publicznego szablonu, README, komendy startupu ani GitHuba. **.env** jest ignorowany przez Git.

Baza i sieć

API tworzy sześć tabel w już istniejącej bazie: users, sessions, records, members, invites, operations. Konto potrzebuje CREATE oraz SELECT/INSERT/UPDATE/DELETE. Port 3306 bazy i port 9853 API to dwa odrębne połączenia. HTTPS domeny nie zabezpiecza automatycznie MySQL; certyfikat CA bazy jest osobną konfiguracją.

Nie ma automatycznej migracji SQLite do MySQL. Nie uruchamiaj kilku procesów backendu na tej samej bazie w ramach tego projektu; kontrola współbieżności zakłada pojedynczy proces. Ogranicz zdalny dostęp do bazy według możliwości hostingu. **remote: %** dopuszcza wszystkie hosty przy poprawnym uwierzytelnieniu i nie zastępuje hasła ani kontroli sieci.

Uruchomienie i sprawdzenie

1. W GitHub Desktop wyślij sprawdzone commity przez Push origin.
2. Sprawdź połączenie Wispbyte z repozytorium i gałęzią main. Dostęp do repozytorium prywatnego ustaw w panelu.
3. Zapisz konfigurację startupu i Environment; sprawdź pełną nazwę użytkownika bazy.
4. Uruchom serwer. Oczekiwany log: **TripSync API na porcie 9853**.
5. Sprawdź **https://tripsync.wisp.uno/health**; aktualne API raportuje **version: 1.1.0**, Android ma 1.2.0.
6. Utwórz konto prezentacyjne i wyjazd, odczytaj na drugim koncie po zaproszeniu, zrestartuj backend i sprawdź trwałość.

Health potwierdza działający proces. Zapis, odczyt i restart weryfikują dodatkowo pracę bazy. Zwracanie 404 dla **/** nie oznacza błędu: backend jest API, nie stroną internetową. Nie wyłączaj sprawdzania certyfikatów, aby korzystać z adresu IP zamiast domeny.

Aktualizacje, kopie i błędy

Domyślny startup robi git pull, instaluje requirements i uruchamia main.py. Kroki w pokazanej komendzie są rozdzielone średnikami: błąd pull/pip nie musi zatrzymać dalszego startu, dlatego sprawdzaj cały log. Aktualizacja kodu nie powinna zastępować **.env** ani kasować bazy.

MySQL kopiuj przez eksport panelu lub odpowiednio skonfigurowany mysqldump; **backend/backup.py** jest wyłącznie dla SQLite. Kopie przechowuj poza serwerem. Przy błędzie MySQL sprawdź kolejno host, port, pełny login, nazwę bazy, hasło i dopuszczenie połączenia z serwera API. Nie publikuj zrzutu z widocznym hasłem.

Bezpieczeństwo i obsługa błędów

Android

Sesja jest szyfrowana AES-GCM. Klucz powstaje w Android Keystore; zaszyfrowane dane i IV znajdują się w prywatnych SharedPreferences. Hasło logowania nie jest zapisywane przez rememberSaveable. Baza Room znajduje się w prywatnym katalogu aplikacji, bez dodatkowego szyfrowania SQLCipher. Backup aplikacji i transfer danych są wyłączone w konfiguracji.

W release obowiązuje HTTPS. Debug dopuszcza HTTP tylko dla lokalnych adresów testowych. Zmiana serwera jest możliwa bez aktywnej sesji, aby nie wysłać tokenu do innego API. Wylogowanie czyści bazę, kolejkę, konflikty i sesję lokalną; niewysłane zmiany zostaną utracone.

Backend

Hasła: PBKDF2-HMAC-SHA256, losowa sól, 260 000 iteracji i porównanie stałoczasowe. Token sesji zawiera 48 losowych bajtów przed kodowaniem; backend przechowuje jego hash SHA-256. Sesja wygasa po 7 dniach. Limiter rejestracji/logowania: 30 prób na adres IP w 5 minut, w pamięci pojedynczego procesu.

Każda operacja sprawdza sesję, dostęp do wyjazdu, rolę, payload i wersję. Organizator zarządza wyjazdem, budżetem, zaproszeniami i uczestnikami. Użytkownik głosuje za siebie i edytuje wyłącznie własne spłaty. Parametry SQL są przekazywane jako parametry, nie przez sklejanie danych użytkownika z zapytaniem.

Hosting i dane

Hasło MySQL znajduje się wyłącznie w Environment albo **.env** serwera. **.gitignore** wyklucza sekrety, bazy, kopie i klucze podpisu. HTTPS aplikacji zabezpiecza kanał Android → API; nie zapewnia automatycznie TLS kanału API → MySQL. Ustawienie **TRIPSYNC_MYSQL_SSL_CA** włącza weryfikację TLS bazy, jeśli hosting udostępnia odpowiedni certyfikat.

Nagłówki adresu klienta są akceptowane tylko od jawnie zaufanych proxy. Kopie bazy muszą być przechowywane poza tym samym serwerem; **backend/backup.py** obsługuje SQLite, a MySQL wymaga eksportu narzędziem hostingu lub **mysqldump**.

Mapa i zewnętrzne API

Leaflet jest lokalnym zasobem. WebView nie ma mostka JavaScript do Androida, blokuje dostęp do plików/treści, mieszany HTTP i niepożądaną nawigację. Nazwy markerów są kodowane przez Gson oraz wstawiane jako textContent. CSP ogranicza źródła. Android nie pobiera GPS telefonu; współrzędne miejsca są przekazywane dostawcy mapy i prognozy. Klient Open-Meteo nie otrzymuje tokenu TripSync.

Obsługa błędów

Walidacja daje opis pola lub reguły przed zapisem. Błędy HTTP są mapowane z komunikatu serwera, a brak sieci pozostawia lokalne dane i kolejkę. Konflikt nie jest nadpisywany automatycznie. Mapa pokazuje błąd i ponowienie; pogoda zachowuje cache z datą. Backend zwraca ujednolicone JSON **message**, bez ujawniania sekretów konfiguracji.

Ograniczenia

To podstawowe zabezpieczenia projektu dydaktycznego, bez przeprowadzonego audytu bezpieczeństwa. Nie ma odzyskiwania hasła, weryfikacji e-mail, samodzielnego usuwania konta ani monitoringu produkcyjnego. Urządzenie offline może nadal wyświetlać kopię danych po odebraniu dostępu na serwerze, aż do udanej synchronizacji. Pełna lista: **ograniczenia.md**.

Testowanie i wyniki wersji 1.2

Wykonane sprawdzenia

Wyniki zachowane z 05.10.2026. To raport z wykonanych przebiegów, nie plan przyszłych testów.

Grupa	Liczba	Niepowodzenia
Backend Python	37	0
Jednostkowe Android / Kotlin	21	0
Android: UI, integracja REST i cache pogody	20	0
Razem, unikalne testy	78	0

Dodatkowo powtórzono wybór daty i godziny na ekranie 320 dp ze skalą czcionki 1.3 oraz pokaz głównych widoków ze sprawdzeniem kontrastu systemowych pasków w ciemnym motywie. Powtórzenia nie zwiększają liczby 78. Budowanie zakończyło się powodzeniem. Lint: 0 błędów i 16 ostrzeżeń wyłącznie o dostępnych aktualizacjach Gradle oraz zależności.

Po poprawce katalogu kompilacji na macOS ponownie wykonano świeże budowanie i 21 testów jednostkowych, a następnie powtórne budowanie bez dodatkowej konfiguracji ścieżki. Oba przebiegi zakończyły się powodzeniem; nowe pliki nie miały kopii **2.class** / **3.class**.

Środowisko i dowody

API 29 / Android 10, ARM64, 1080 × 2400 px, 420 dpi; wariant kompaktowy 840 × 1470 px i font scale 1.3. Kompilacja: SDK 36.1, AGP 9.1.1, Gradle 9.3.1, JDK 21. Backend testów: Python 3.13 i osobna lokalna SQLite. Testy integracji nie tworzyły kont na publicznym serwerze.

- Wyniki maszynowe: **docs/test-results-1.2.json**.
- JUnit XML, logi budowania i Lint HTML/XML/TXT: **output/reports/ui-1.2/**.
- Zrzuty działającego interfejsu: **docs/screenshots-1.2/**.
- Poprzednie wyniki w **docs/test-results.json**, **docs/screenshots/** oraz raportach poza **ui-1.2/** są archiwalne.

Zakres testów jednostkowych

Odczyt kwot, przecinek/kropka, precyzja, granice i przepełnienie; dokładny podział groszy, walidacja udziałów, salda i przelewy. Testy obejmują 200 deterministycznych losowych zestawów rozliczeń, spłaty częściowe, całkowite i nadpłatę. Sprawdzono budżety, współrzędne, daty i godziny. Testy dat weryfikują polskie formatowanie oraz zachowanie dnia przy zmianach strefy i przejściu czasu letniego.

Zakres backendu

Konta, hashowanie, sesje, logowanie i błędne dane; uprawnienia, zaproszenia, idempotencja, konflikty, tombstone i głosowanie; wydatki, kategorie, budżety, spłaty i historia. Dodatkowo kopia SQLite, kontrola proxy, WSGI, kontrakt HTTP, konfiguracja **.env** i adapter MySQL. Testy adaptera MySQL używają sterownika zastępczego; nie są pełnym automatycznym testem usługi MySQL Wispbyte.

Zakres urządzeniowy

CRUD wydatku i salda, walidacja formularza, głosowanie, budżet, spłata, miejsca, historia i główne ekrany. Odtwarzanie aktywności zachowuje formularz i dane. Regresje 1.2 obejmują wybór daty/godziny, ochronę przed odrzuceniem formularza, filtr kosztów po przełączeniu zakładki oraz zachowanie udziałów przy dołączeniu nowej osoby.

Integracja REST używa dwóch niezależnych repozytoriów i kont: zapis, kolejka, synchronizacja, konflikt z wyborem obu wersji, usunięcie i cofnięcie dostępu. Test cache pogody sprawdza zachowanie zapisanej prognozy po błędzie sieci.

Jak powtórzyć testy

```
python3 -m unittest discover -s backend -v
./gradlew :app:testDebugUnitTest :app:lintDebug
```

Do testów urządzeniowych uruchom dedykowany emulator oraz osobny serwer na porcie 8080:

```
python3 backend/server.py --db /tmp/tripsync-test.sqlite
```

W drugim terminalu:

```
./gradlew :app:connectedDebugAndroidTest
```

Testy UI wylogowują aplikację i tworzą demo, więc nie uruchamiaj ich na urządzeniu z ważnymi danymi użytkownika. Integracja ustawia adres **http://10.0.2.2:8080/**. Przy kilku emulatorach ogranicz wybór do testowego urządzenia zmienną **ANDROID_SERIAL**; nie testuj przypadkowo aktywnego telefonu.

Gradle może usunąć APK po testach. Do zachowania screenów można osobno zainstalować oba APK i uruchomić runner:

```
adb -s EMULATOR shell am instrument -w -e class 'pl.tripsync.TripSyncUiTest#mainScreensAndRealScreenshots' pl.tripsync.test/androidx.test.runner.AndroidJUnitRunner
adb -s EMULATOR pull /sdcard/Android/data/pl.tripsync/files/screenshots/. /tmp/tripsync-screenshots/
```

Granice weryfikacji

Nie wykonano pełnych testów na nowszych Androidach, tablecie i fizycznym telefonie, audytu TalkBack, bezpieczeństwa ani testów obciążenia. Test **scenario.recreate()** nie jest testem zabicia procesu przez system. Starszy raport opisuje dodatkowe ręczne force-stop i odczyt demo; nie rozciągamy tego wyniku na wszystkie scenariusze wersji 1.2. Wyniki nie dowodzą braku każdego możliwego błędu.

Zgodność z wytycznymi zajęć Android

Ocena dotyczy kodu Android 1.2.0, backendu API 1.1.0 i zapisanych wyników testów. Nie jest przyznaniem punktów ani gwarancją oceny prowadzącego.

Wymagania techniczne

Wymaganie	Status	Dowód / uwaga
Java lub Kotlin	Spełnione	Kotlin w app/src/main/java
Android Studio i aktualne stabilne SDK	Częściowo	Android Studio działa; SDK 36.1. Aktualizacja do 37 pozostaje do wykonania
MVVM lub Clean Architecture	Spełnione w wariancie MVVM	TripViewModel, Repository, domain, data, ui
Compose lub XML / Material 3	Spełnione	Compose i Material 3
DBMS	Spełnione	Room/SQLite oraz adapter backendu MySQL
REST / Retrofit	Spełnione	TripApi, TravelRepository, API Python
Coroutines / Flow	Spełnione	ViewModel, DAO Flow i suspend operacje
Stan i nawigacja	Spełnione	StateFlow, rememberSaveable i Navigation Compose
Ustawienia	Spełnione	DataStore: motyw i adres API
Obsługa błędów	Zaimplementowane	Walidacja, komunikaty, kolejka i konflikty; bez wyczerpującego testu awarii każdej bazy
Jednostkowe i UI	Spełnione	21 jednostkowych i 20 urządzeniowych w zapisanym przebiegu
Podstawowe bezpieczeństwo	Zaimplementowane	Keystore, HTTPS, backendowe role i walidacja; bez audytu
Git	Spełnione lokalnie	Historia i origin; dostęp prowadzącego do repozytorium do potwierdzenia
Dokumentacja	Przygotowana wersja robocza	README, Markdown, PDF, diagramy; dane autorów do uzupełnienia

Funkcjonalne i niefunkcjonalne

CRUD, walidacja, ponad cztery ekrany, nawigacja, trwałość i obsługa błędów są zaimplementowane. Rozszerzenia obejmują wiele kont, współpracę offline, konflikty, rozliczenia, budżety, historię, mapy i pogodę.

Responsywność sprawdzono na telefonicznym emulatorze oraz dla pickerów na małym ekranie ze skalą czcionki 1.3. ViewModel i rememberSaveable wspierają odtworzenie aktywności; istnieje test tego scenariusza. Opisy kontrolek i motywy wspierają dostępność, ale pełne sprawdzenie TalkBack, tabletu i najnowszego Androida pozostaje do wykonania. Czytelność i modularność kodu należy ocenić w przeglądzie; nie utożsamiamy samych folderów z idealną architekturą.

Kryteria oceny

Kryterium	Maksimum	Materiał do oceny
Funkcje	25	CRUD, plan, koszty, współpraca, mapa i pogoda

Kryterium	Maksimum	Materiał do oceny
Architektura i kod	15	MVVM, domena Money/Validation, Repository, transakcje i synchronizacja
UI i UX	10	Material 3, formularze, pickery, dwa motywy i błędy
Baza i dane	10	Room, MySQL/SQLite, schematy, indeksy, trwałość i kolejka
API / urządzenie	10	REST, Open-Meteo, mapa, powiadomienia i WorkManager
Błędy i bezpieczeństwo	10	Role backendu, Keystore, walidacja, konflikty i komunikaty
Testy	10	21 jednostkowych Android, 20 urządzeniowych oraz 37 backendu
Dokumentacja	5	Cel, wymagania, diagramy, instrukcje, screeny, wyniki, repo i APK
Prezentacja i obrona	5	Samodzielny pokaz i wyjaśnienie rozwiązań przez studentów
Razem	100	Punkty przyznaje prowadzący po weryfikacji

Pakiet wymagany od studentów

Cel i zakres: **wymagania.md**; specyfikacja: ten sam dokument. Diagram architektury: **architektura.md** i SVG. UML przypadków użycia: **przypadki-uzycia.md** i SVG. Bazy: **baza-danych.md**, schemat Room i backend. Widoki: **funkcje.md** oraz **screenshots-1.2/**. Biblioteki: **technologie.md**. Instalacja: **uruchomienie.md**. Testy: **testy.md**, JSON i raporty. Repozytorium: https://github.com/Yamoteusz/TripSync. APK: lokalna paczka lub artefakt udanego workflow. Całość podsumowuje PDF.

Nazwiska i rzeczywisty wkład należy uzupełnić w **AUTHORS.md**, a formę indywidualnego oddania przez trzy osoby uzgodnić z prowadzącym. Zakres nie wymaga płatności ani publikacji w Google Play.

Źródło planowanej aktualizacji SDK: Android 17 / API 37. Obecna dokumentacja nie udaje, że aktualizacja została wykonana.

Foldery i aktualizacje projektu

Projekt ma jedną aplikację Android i odrębny backend. Podział folderów umożliwia osobną pracę nad funkcjami; wszystkie katalogi razem tworzą działający projekt. Nie należy wgrywać samych ekranów bez wspólnych zależności.

Folder	Zakres
backend/	API, konta, kontrola dostępu, synchronizacja i testy Python
app/src/main/java/pl/tripsync/domain/	Modele, walidacja i algorytm rozliczeń
app/src/main/java/pl/tripsync/data/	Room, kolejka, konflikty, Retrofit, DataStore i Keystore
app/src/main/java/pl/tripsync/ui/screens/auth/	Logowanie i rejestracja
app/src/main/java/pl/tripsync/ui/screens/trips/	Lista wyjazdów
app/src/main/java/pl/tripsync/ui/screens/trip/	Podsumowanie i połączenie zakładek
app/src/main/java/pl/tripsync/ui/screens/plan/	Harmonogram
app/src/main/java/pl/tripsync/ui/screens/tasks/	Zadania i pakowanie
app/src/main/java/pl/tripsync/ui/screens/ideas/	Propozycje i głosowanie
app/src/main/java/pl/tripsync/ui/screens/money/	Koszty, kategorie, salda i CRUD spłat
app/src/main/java/pl/tripsync/ui/screens/budget/	Budżet i limity kategorii
app/src/main/java/pl/tripsync/ui/screens/activity/	Historia zmian
app/src/main/java/pl/tripsync/ui/screens/places/	Mapa, miejsca, wyszukiwanie i pogoda
deploy/	Hosting WSGI oraz alternatywnie Docker/Caddy
app/src/main/java/pl/tripsync/ui/screens/editor/	Formularze dodawania i edycji
app/src/main/java/pl/tripsync/ui/screens/members/	Uczestnicy i zaproszenia
app/src/main/java/pl/tripsync/ui/screens/conflicts/	Rozstrzyganie konfliktów
app/src/main/java/pl/tripsync/ui/screens/settings/	Ustawienia i sesja
app/src/test/ i **app/src/androidTest/**	Testy domeny, interfejsu i synchronizacji
docs/	Dokumentacja, diagramy i screeny
scripts/	Powtarzalne budowanie materiałów końcowych

Zasada aktualizacji w Git

Commit powinien opisywać rzeczywistą zmianę. Nowe funkcje, poprawki i testy dodawaj w miarę ich realizacji. Zmiany w powiązanych folderach można zatwierdzać razem, aby każdy etap zachował spójne zależności.

Przykład kolejnej rzeczywistej poprawki harmonogramu:

```
git switch -c codex/plan-poprawka
```

Po wprowadzeniu i sprawdzeniu zmiany:

```
git add app/src/main/java/pl/tripsync/ui/screens/plan
git commit -m "fix(plan): popraw wyświetlanie kolejności punktów"
```

Jeżeli poprawka wymaga zmiany repozytorium albo testu, dodaj je w tym samym commicie. Repozytorium origin jest skonfigurowane jako https://github.com/Yamoteusz/TripSync. Zatwierdzone zmiany wysyłaj przez Push origin w GitHub Desktop. Faktyczny wkład trzech osób uzupełnij w AUTHORS.md.

Zawartość paczki

Paczka do oddania obejmuje źródła, Gradle Wrapper, backend, dokumentację, screeny, APK i raporty. Nie zawiera prywatnych sesji, baz testowych, lokalnych ścieżek SDK ani cache narzędzi. Git bundle pozwala odtworzyć lokalną historię również bez zdalnego serwera.

Zakres dla trzech osób po rozbudowie

Osoba	Główne moduły	Weryfikacja
1	Konta, uprawnienia, API, WSGI/hosting, historia i kopie bazy	Testy backendu, idempotencja, trwałość i instrukcja wdrożenia
2	Plan, zadania, pomysły, mapa wspólnych miejsc, Open-Meteo i cache prognozy	Formularze, błędy API, UI i dokumentacja map/pogody
3	Wydatki i kategorie, budżety, spłaty, algorytm sald, Room/synchronizacja	Testy domeny i integracja między kontami

To propozycja odpowiedzialności do dalszej pracy i obrony. Każda osoba powinna rozumieć swój zakres oraz wspólną architekturę.

GitHub Desktop i aktualizacje zespołu

Repozytorium jest skonfigurowane jako Yamoteusz/TripSync, gałąź main. Zachowujemy istniejącą historię; nie trzeba ponownie publikować repozytorium pod inną nazwą.

Lokalny katalog

W GitHub Desktop wybierz File → Add Local Repository i katalog zawierający README, app, backend i **.git**. Nie dodawaj osobno **app** ani rozpakowanego ZIP-a. Gdy nowe commity już istnieją lokalnie, kliknij **Push origin**. Gdy dopiero zmieniłeś pliki, najpierw wykonaj commit z opisem rzeczywistej zmiany.

Praca trzech osób

Przed pracą Fetch origin i Pull origin. Przy równoległym rozwijaniu używajcie osobnych gałęzi i pull requestów. Przykład nazwy: **codex/plan-kalendarz**. Włączcie dostęp pozostałych osób oraz prowadzącego, jeśli repozytorium jest prywatne. Podział odpowiedzialności opisuje **organizacja-pracy.md**, a faktyczny wkład należy wpisać do **AUTHORS.md**.

Commit zapisuje stan lokalnie, push publikuje go na GitHubie, a aktualizacja Wispbyte pobiera zdalny kod. Nie tworzymy fikcyjnej historii przez rozłożone w czasie wysyłanie gotowych folderów.

GitHub Actions

Workflow **.github/workflows/checks.yml** wykonuje testy backendu, kontrolę składni Python, debug APK, testy jednostkowe Android oraz lint. Publikuje artefakty TripSync-debug i android-reports. Nie uruchamia testów emulatora i nie wdraża automatycznie backendu. Wyniki lokalne dokumentacji nie są dowodem udanego przebiegu Actions; jego stan należy sprawdzić na GitHubie.

Co nie trafia do historii

Bazy i kopie, **.env**, hasła, tokeny, klucze podpisu, local.properties, cache, pliki budowania i APK. **gradle/gradle-daemon-jvm.properties** pozostaje lokalny. Przykłady env są szablonami bez hasła; nie są automatycznie odczytywane. PDF, screeny i raporty dokumentacyjne są wersjonowane.

Aktualizacja hostingu

Po Push origin uruchom lub zrestartuj Wispbyte zgodnie z wispbyte.md. Zachowaj domyślny startup uruchamiający main.py oraz konfigurację Environment. Publikacja plików app nie instaluje nowego Androida na telefonie: aplikację aktualizuje się przez Run albo APK.

Prezentacja i obrona projektu

Scenariusz pokazu, około 10-12 minut

Etap	Co pokazać	Co wyjaśnić
1. Cel, 1 minuta	Lista wyjazdów i podsumowanie	Problem wspólnego planu i kosztów; zakres dla trzech osób
2. Plan i formularze, 2 minuty	Nowy punkt, kalendarz, zegar, zadanie i propozycja	CRUD, walidacja oraz zachowanie niezapisanego formularza
3. Finanse, 2 minuty	Wydatek, własne udziały, budżet, saldo i spłata	Kwoty całkowite w groszach, reszta przy podziale, spłata jako deklaracja
4. Offline i współpraca, 3 minuty	Dwa konta, zapis offline, kolejka, synchronizacja i konflikt	Room jako źródło danych, wersjonowanie oraz idempotencja
5. API i urządzenie, 1 minuta	Mapa, pogoda, cache lub lokalne przypomnienie	Retrofit, zewnętrzne API, obsługa błędów i ograniczenia Androida
6. Dowody, 1-3 minuty	Diagramy, testy, Git i APK	Architektura, rzeczywisty wkład i granice weryfikacji

Demo wystarcza do pokazania lokalnego CRUD. Współdzielenie, autorzy historii i konflikty wymagają dwóch rzeczywistych kont na tym samym API. Używaj kont prezentacyjnych, nie danych prywatnych. Sprawdź internet i dostępność backendu przed zajęciami; miej APK oraz lokalny wariant serwera przygotowane wcześniej.

Przykład do rozliczeń

Anna płaci 720 PLN za nocleg, Bartek 180 PLN za bilety; oba koszty dotyczą Anny, Bartka i Celiny w równych udziałach. Każda osoba ma koszt 300 PLN. Salda: Anna +420 PLN, Bartek -120 PLN, Celina -300 PLN. Propozycje przelewów: Bartek → Anna 120 PLN i Celina → Anna 300 PLN. Po zapisaniu spłaty Bartka pozostaje przelew Celiny. Pokaż, że suma sald nadal wynosi zero.

Pytania, na które każdy powinien umieć odpowiedzieć

- Gdzie znajduje się stan aplikacji i czym różni się ViewModel od repozytorium?
- Dlaczego ekran czyta Room zamiast bezpośrednio wynik Retrofit?
- Co dzieje się z zapisem bez internetu i co oznacza **baseVersion**?
- Jak wykrywa się konflikt i dlaczego ponowienie operacji nie tworzy duplikatu?
- Dlaczego pieniądze są **Long** w groszach, a nie **Double**?
- Które relacje bazy są FK, a które są zapisane w JSON i kontrolowane przez kod?
- Dlaczego sprawdzenie roli wyłącznie w UI nie byłoby wystarczające?
- Jak jest przechowywany token i jaka jest różnica między HTTPS API a TLS MySQL?
- Które testy są jednostkowe, a które wymagają emulatora oraz backendu?
- Co zostało zweryfikowane, a czego nie obejmuje obecny raport?

Oddanie przez każdego studenta

Uzgodnijcie z prowadzącym sposób przekazania wspólnego repozytorium i dokumentacji przez trzy osoby. Uzupełnijcie **AUTHORS.md** na podstawie faktycznej pracy. Materiały nie są gwarancją punktacji; ocena obejmuje samodzielny pokaz i znajomość rozwiązań.

Ograniczenia i dalsze prace

Stan: Android 1.2.0, backend API 1.1.0. Poniższe elementy nie są przedstawiane jako już ukończone funkcje.

Granice bieżącej aplikacji

- Jedna waluta na wyjazd: PLN, EUR albo USD. Nie ma przeliczania kursów.
- Maksymalnie 100 uczestników. Rozliczenia zachowują dokładność grosza/centa. Algorytm tworzy co najwyżej n-1 przelewów dla n osób z niezerowym saldem, ale nie gwarantuje globalnie najmniejszej liczby przelewów.
- Spłata jest deklaracją nadawcy; aplikacja nie potwierdza jej w banku i nie wykonuje płatności.
- Synchronizacja pobiera pełny snapshot, jest ręczna lub okresowa przez WorkManager, bez aktualizacji w czasie rzeczywistym. Usunięte uprawnienia są egzekwowane na serwerze natychmiast; urządzenie offline zachowuje wcześniej pobraną kopię do udanej synchronizacji.
- Okresowa synchronizacja i lokalne przypomnienia podlegają ograniczeniom Androida. Nie są alarmami dokładnymi.
- Wylogowanie czyści lokalną bazę i niewysłane zmiany. Demo nie jest eksportowane do prawdziwego konta.
- Mapa potrzebuje internetu do kafelków. Brak map offline i GPS telefonu. Wyszukiwanie obejmuje miejscowości; inne punkty można podać współrzędnymi.
- Prognoza obejmuje najbliższe 7 dni, nie dowolny termin przyszłego wyjazdu. Cache jest oznaczany datą.
- Historia pokazuje 200 najnowszych wpisów; brak paginacji i cofania zmian.
- Backend działa jako jeden proces z serializowanymi zapisami; nie zweryfikowano wieloprocesowego wdrożenia ani obciążenia.
- Brak odzyskiwania hasła, weryfikacji e-mail i samodzielnego usuwania konta. Brak czatu, załączników, skanowania paragonów, QR, szablonów oraz eksportu PDF/ICS z poziomu aplikacji. Udostępnianie tekstowego planu już działa.
- Nie ma płatności, premium, donate ani reklam. Zakres pozostaje projektem na zajęcia Android.

Zadania przed oddaniem

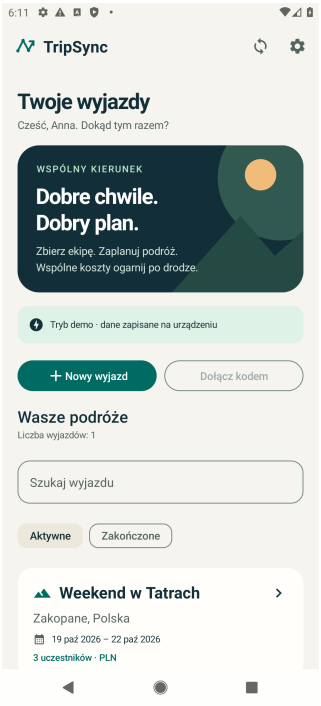
1. Uzupełnić nazwiska, numery albumów i rzeczywisty wkład w **AUTHORS.md** oraz ponownie wygenerować PDF.
2. Zaktualizować SDK 36.1 do 37, jeśli prowadzący wymaga najnowszego stabilnego wydania; przetestować zachowanie po aktualizacji. Obecna dokumentacja celowo podaje używane SDK.
3. Sprawdzić aplikację na nowszym Androidzie, fizycznym telefonie, tablecie, przy obrocie i z TalkBack. Test odtworzenia aktywności nie jest dowodem dla każdego scenariusza zmiany konfiguracji.
4. Zweryfikować dostęp prowadzącego do repozytorium i aktualny APK, a także powtórzyć scenariusz dwóch kont na wybranym serwerze.
5. Przejrzeć jakość kodu: granice odpowiedzialności ViewModel/repozytorium, powtarzalne formularze, formatowanie i logikę domenową. Podział na foldery nie zastępuje przeglądu architektury.

Kolejne funkcje bez płatności

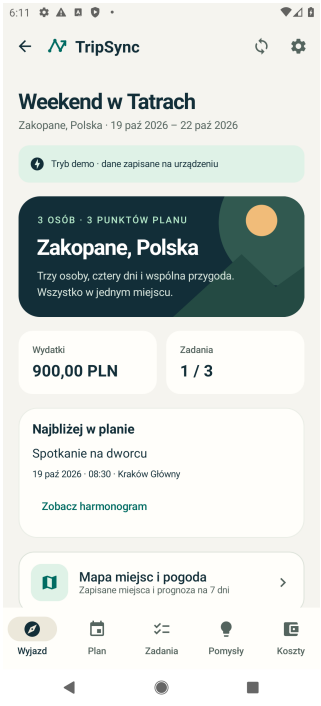
Szablony i kopiowanie wyjazdów, zaproszenia QR, eksport kalendarza, komentarze przy punktach planu i powiadomienia grupowe. To propozycje następnych wersji, bez obietnicy wykonania w 1.2.0.

Widoki aplikacji / 1

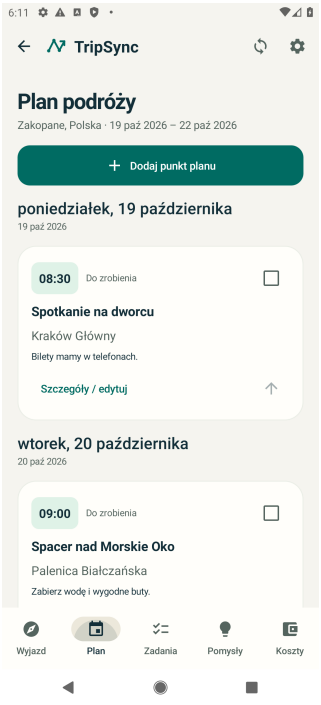
Rzeczywiste zrzuty z testów wersji 1.2.0, 05.10.2026. Dane pochodzą z demo i izolowanego środowiska testowego.



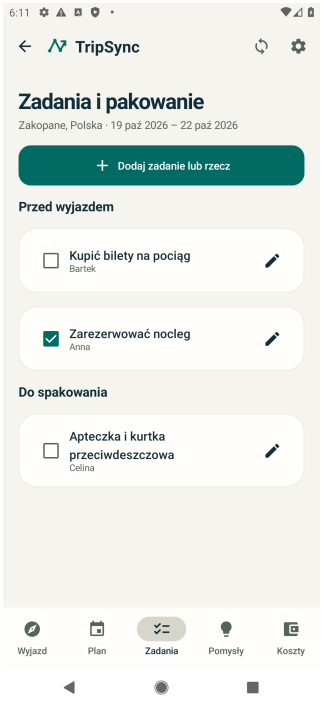
Wyjazdy



Podsumowanie



Harmonogram

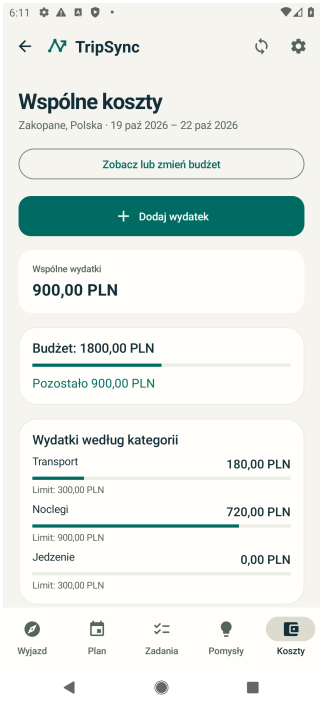


Zadania i pakowanie

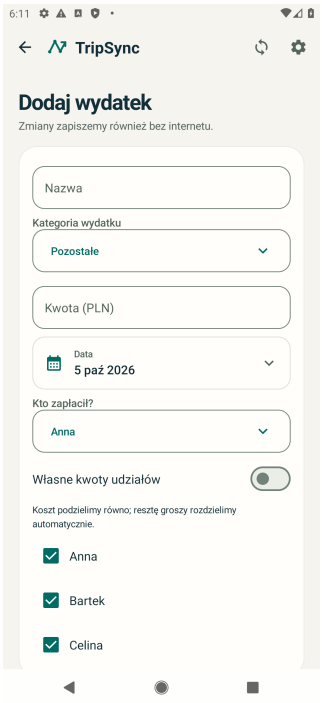
Widoki aplikacji / 2



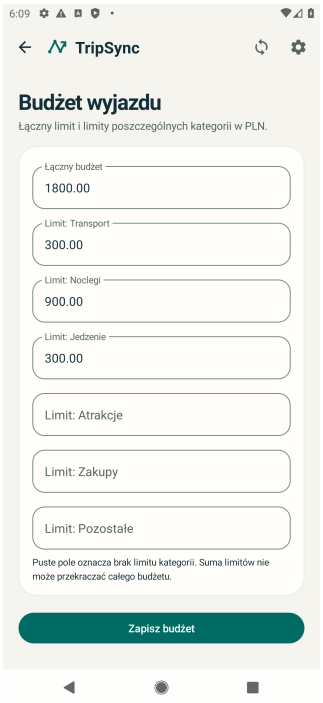
Głosowanie



Koszty i rozliczenia

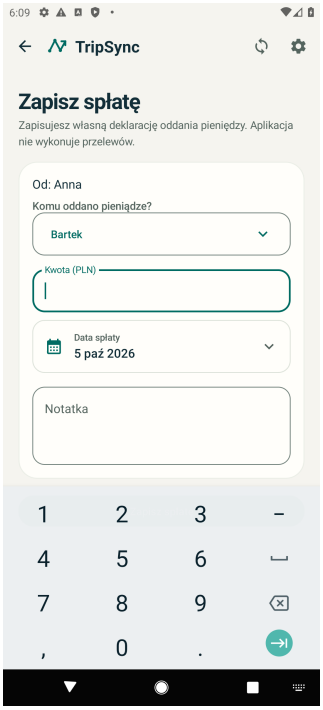


Formularz wydatku

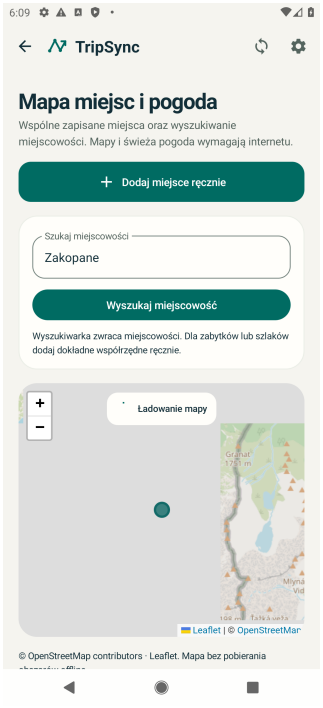


Budżet i kategorie

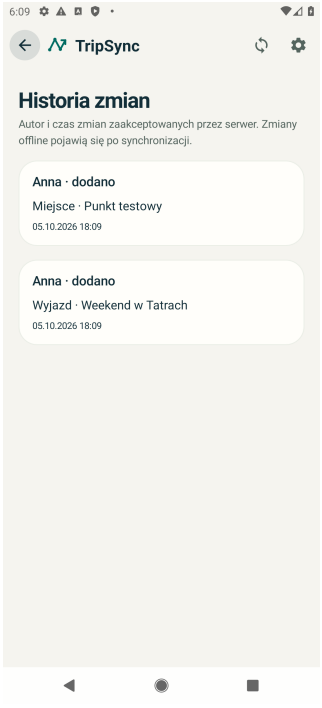
Widoki aplikacji / 3



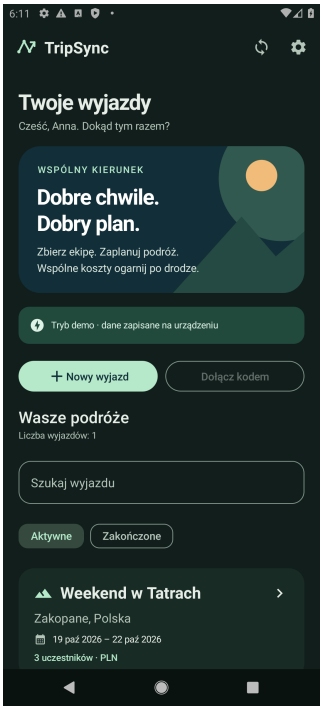
Spłata



Mapa miejsc

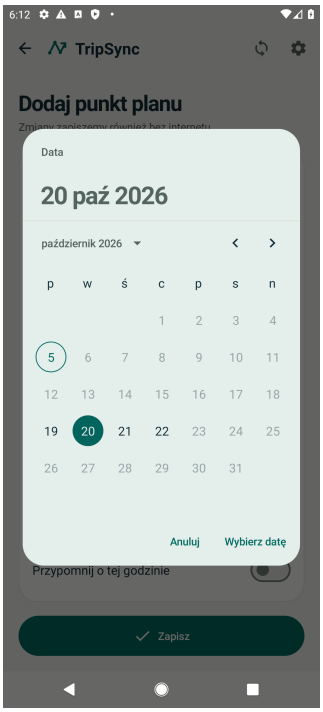


Historia zmian



Motyw ciemny

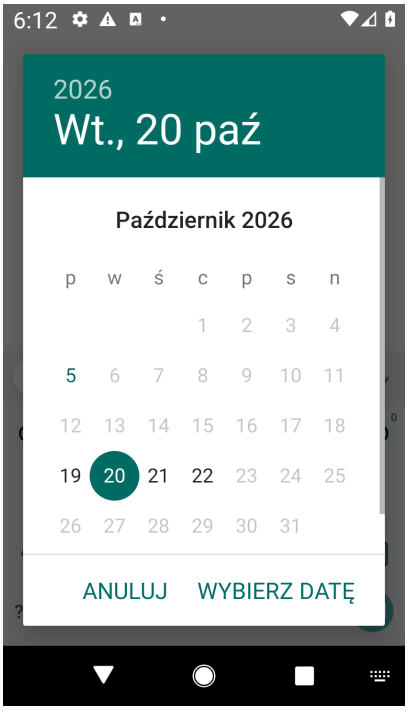
Widoki aplikacji / 4



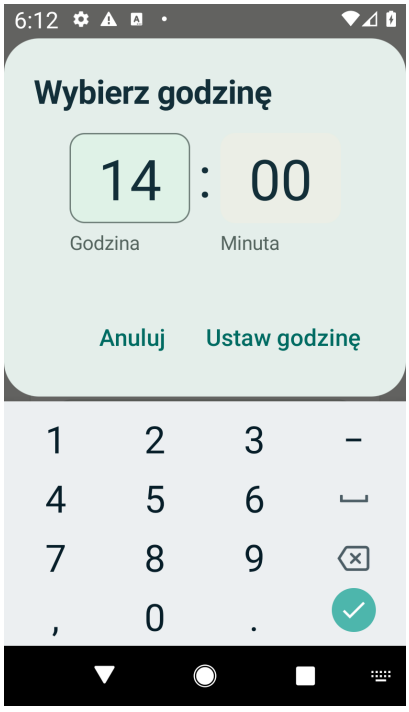
Wybór daty



Wybór godziny



Kalendarz: mały ekran



Zegar: mały ekran